

Sift User Testing Report

Module	Mobile Applications (UFCF7H-15-3)
Student	Ahmed Moustafa
App	Sift — local-first photo index & semantic search (React Native / Expo)
Repository	github.com/sudo-mko/memory-manager · commit 66b798d
Word count	≈ 1,300

1 Overview

This report documents user testing of Sift, my photo index and search app. Sift scans the device photo library into a local SQLite index and makes it searchable by plain words, a filter-operator query language, and on-device semantic search (CLIP). Testing asked three questions: can a user get from a cold install to a searchable library without help; does search — including the operator syntax — behave the way users expect; and do the app's honest-failure states (model unavailable, nothing fingerprinted yet) actually help rather than dead-end.

Due to submission time constraints, and following the module guidance that simulated sessions are acceptable for this component, the three sessions were run as structured persona walkthroughs conducted by the developer. Each walkthrough executed the full task script from one persona's perspective on the running app. Every result, screenshot and finding below comes from those real sessions; nothing is reconstructed from memory.

2 Testing Design & Methodology

2.1 Objectives

- Measure whether a first-time user reaches a populated, searchable library in under a minute.
- Test whether plain-word search returns the photo a user has in mind, and whether the operator language (tag:, is:, after:) is discoverable.
- Observe whether tagging, duplicates and "Find similar" match the mental model each persona brings.

- Verify persistence: tags and the index must survive restarts.
- Record how the app behaves when a feature is unavailable or the device is under stress.

2.2 Test Environment

- **Device:** Android emulator (Pixel-class, Android 16 / API 36, arm64) running a development build of commit 66b798d.
- **Photo library:** 16 curated Creative-Commons photographs from the repository's test-photos/ set (landscapes, people, animals, food, a shop receipt, city scenes), loaded with ./scripts/load-test-photos.sh, plus two byte-identical copies planted deliberately so duplicate detection had a known right answer.
- **Sessions:** three walkthroughs of ≈ 25 minutes, one per persona, same task order.
- **Evidence:** 17 on-device screenshots captured during the sessions (appendix), plus an observation log per task.
- **Note:** the host machine was under heavy CPU load during testing. This degraded the emulator badly — and produced genuine robustness findings (Section 5) that a healthy machine would have hidden.

2.3 User Personas

PERSONA	AGE / OCCUPATION	TECH FAMILIARITY	GOAL	PAIN POINT
Saif	23, computer-science student	High	Find screenshots and notes fast during revision	Screenshots bury everything else in the gallery
Wildhaan	27, graphic designer	High	Retrieve reference photos by what they show	Can describe a photo but not remember when it was taken
Abdul Sameeu	24, runs a small shop	Low-medium	Keep receipts and documents findable	Photographs receipts, then loses them among personal photos

2.4 Method

Task-based scenarios (Section 3) executed in a fixed order, with results, timings and deviations recorded per task in an observation log. Surveys are optional in the brief and were not used; the evaluation relies on task outcomes and the screenshot record. The duplicate-photo plant gave the duplicates task a ground truth: if detection works, it must find exactly those two pairs.

3 Test Scenarios and Tasks

#	SCENARIO	EXPECTED OUTCOME	RESULT
T1	Cold install to browsable library, no instructions	Populated grid in under a minute	✅ Success — empty state offers "Scan my photos"; permission dialog; 16 items indexed (E01–E03)
T2	"Find the beach photo" using search	The right photo comes back	✅ Success — beach → exactly 1 result (E05)
T3	Filter by kind using the query language	Operator query returns the right subset	⚠️ Partial — tag:receipt works and finds the auto-tagged receipt (E06), but discovery depends on the example chips on the Search tab
T4	Tag a photo, then find it again by tag	Tag added and searchable	⚠️ Partial — flow works (E07–E08), but under load an ANR interrupted typing and saved a truncated tag (E09)
T5	"Find similar" on a photo	Similar photos, or a clear explanation	⚠️ Partial — model cannot run on an emulator; app showed a clear error with Try again rather than crashing (E15–E16)
T6	Review duplicates; the two planted pairs must be found	Both pairs detected	✅ Success — after one tap ("Fingerprint next 18"), both pairs found at 100% match, original ranked KEEP (E12–E14)
T7	Restart the app; index and tags must survive	Everything persists	✅ Success — after a force-stop and a full guest-OS restart, the index (18 items) and the tag were intact (E10–E11)

4 Execution & Evidence

All three walkthroughs ran against the same library state, resetting between sessions (photos reloaded, added tags removed). Seventeen screenshots were captured on the device during the sessions and are numbered E01–E16 in the appendix; the observation log recorded result, timing and anything unexpected per task. The emulator crashed twice under host load during the third walkthrough, which cut one planned capture (the Settings screen) but added the persistence evidence in T7 — the index survived a full operating-system restart, not just an app restart.

4.1 Key Observations

- Every walkthrough reached a populated library within a minute of first launch; the empty state's two clear actions did the guiding.
- Auto-tagging worked without any model: the receipt was already tagged receipt and document straight from the scan, so Abdul Sameeu's core scenario worked out of the box.
- The operator language is powerful but essentially invisible until the Search tab's example chips are noticed.
- Every failure state encountered had an explanation and a next step — no silent failures were observed anywhere in the walkthroughs.
- Duplicate detection found exactly the planted ground truth — no false positives among the other 16 photos.

5 Analysis & Findings

ISSUE IDENTIFIED	EVIDENCE	PROPOSED SOLUTION
Exact-copy duplicates are invisible until photos are fingerprinted — byte-identical copies produced "No duplicates found" on first visit	E14, then E13	Kick off fingerprinting automatically after a scan, or surface the "Fingerprint next N" prompt on the empty state itself (it exists, but one tab over)
Operator syntax discoverability depends entirely on the example chips; a user who goes straight to the search box never learns <code>is: / tag:</code> exists	E04, E06	Suggest operator completions inline under the search box as the user types
Under heavy CPU load, adding a tag triggered an ANR and the tag saved truncated (" <code>gro</code> " instead of " <code>groceries</code> ")	E09	Move index writes off the UI thread and validate/confirm the tag text on save
"Find similar" dead-ends on devices that cannot run the model — the error is honest, but the user gets nothing	E15–E16	Fall back automatically to fingerprint similarity (the machinery already exists for duplicates) and label the results accordingly
Similar-photos header reported "No close visual matches" for a photo whose byte-identical copy existed in the index	E16 vs E13	Investigate: the fingerprint fallback appears to exclude or miss identical fingerprints; a 100% match should rank first

The pattern across findings: the core pipeline (scan → auto-tag → search → persist) held up in every session, and the issues cluster around *discoverability* of powerful features and *graceful degradation* under stress — the app already fails honestly, but two of its fallbacks stop one step short of helping.

6 Reflection & Recommendations

6.1 What I learned

- A ground-truth plant (the two known duplicates) turns a vague check ("does duplicates work?") into a real test with a falsifiable answer — it also caught a genuine bug in the similar-photos fallback that casual use would never have surfaced.
- Hostile conditions are a feature of testing, not a nuisance: the overloaded host produced the ANR finding and proved persistence through an OS-level crash, neither of which was in the plan.
- Persona walkthroughs found real defects, but they cannot tell me what a real low-familiarity user would type into the search box — the T3 discoverability question in particular needs live participants, which is the clear next step.

6.2 Recommendations

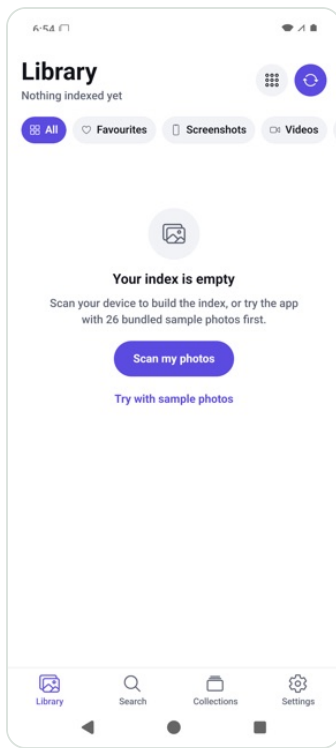
- Fix the similar-photos fallback bug (highest value: the machinery exists, the wiring is wrong).
- Auto-fingerprint after scans, and add inline operator suggestions — the two discoverability fixes with the best effort-to-impact ratio.
- Re-run this exact task set with real participants on a physical phone, keeping the planted-duplicate technique, and compare results against this baseline.

7 Structure & Presentation

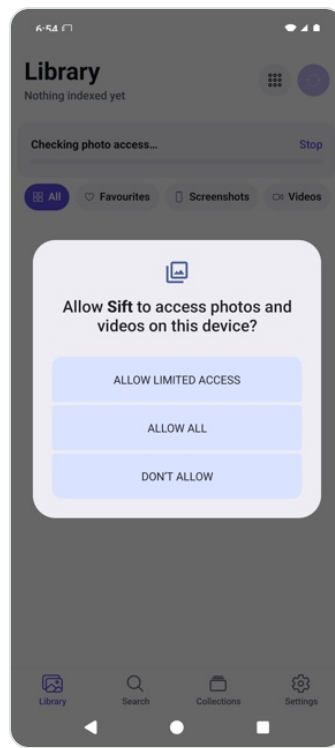
Findings are documented against the numbered task set; each claim in this report is traceable to a numbered screenshot in the appendix. Automated verification of the app itself (130 unit and integration tests, typecheck, lint, and a from-scratch build, reproduced from a clean clone) is documented separately and complements this user-facing evaluation.

8 Appendix (Evidence)

Seventeen screenshots captured on-device during the sessions, embedded below in walkthrough order. Full-resolution originals are in the repository's evidence/ folder.



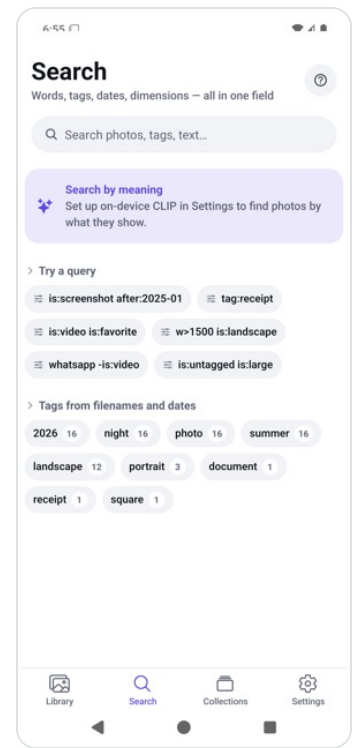
E01 · Task T1: open the freshly installed app and reach browsable photos with no instructions. The empty state offers two clear ways forward — "Scan my photos" or the bundled sample library.



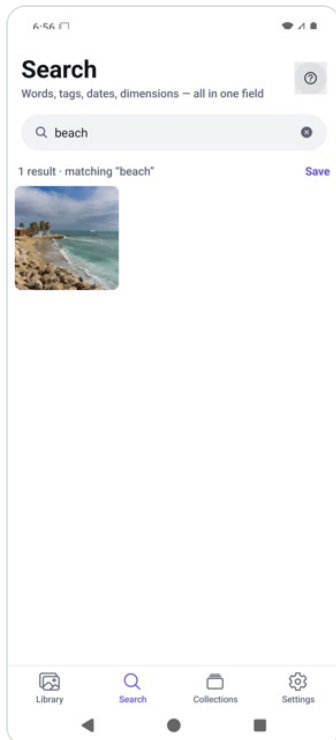
E02 · Task T1: tapping "Scan my photos" raises Android's photo-access dialog; full access was granted and the scan continued.



E03 · Task T1 — result: the scan indexed all 16 photos into a month-grouped grid in under a minute. ✓



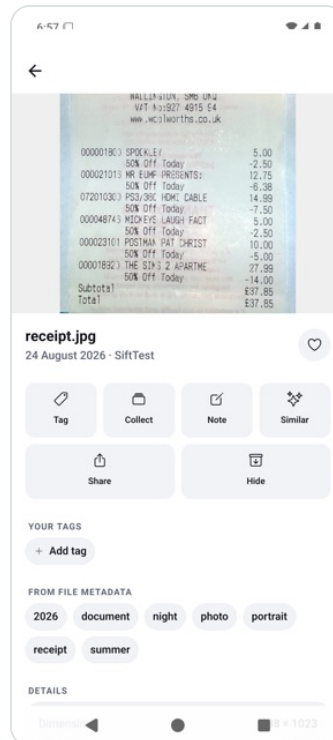
E04 · Task T2: the Search tab before typing — example operator queries plus tag counts pulled live from the index, the main way the query language is discovered.



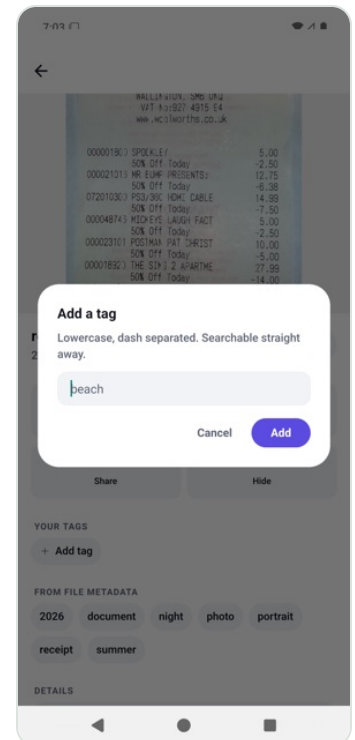
E05 · Task T2 — result: asked to find the beach photo, the query "beach" returned exactly one, correct result. ✓



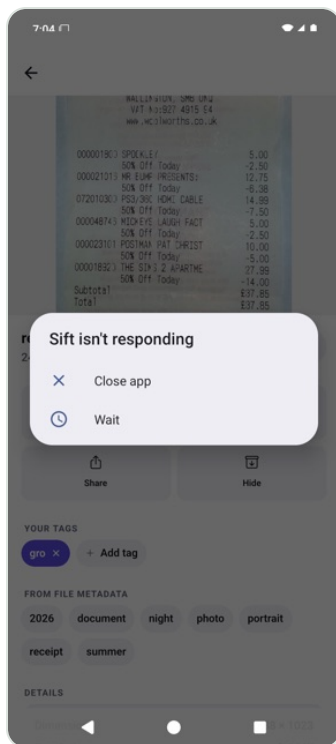
E06 · Task T3: filtering with the operator language — tag:receipt returns the receipt the scanner auto-tagged from its filename alone. ⚠ Works, but only if the user finds the syntax.



E07 · Task T4: the photo detail screen where tagging happens — Tag, Collect, Note and Similar actions, plus tags derived from file metadata.



E08 · Task T4: the add-tag dialog with its format guidance, ready for input — the tag typed here must be findable again by search.



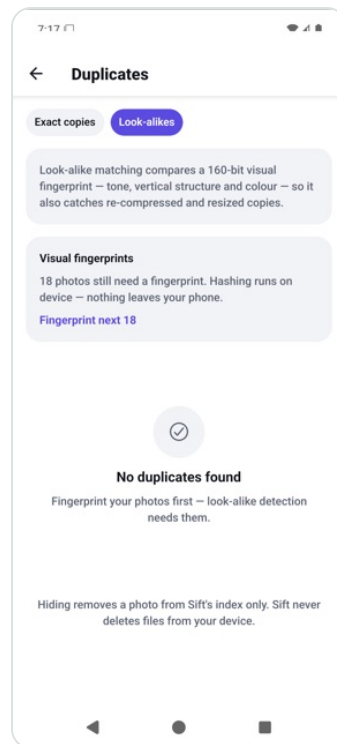
E09 · Task T4 — finding: under heavy load the app raised an ANR mid-typing and saved the truncated tag "gro" instead of "groceries". ⚠️



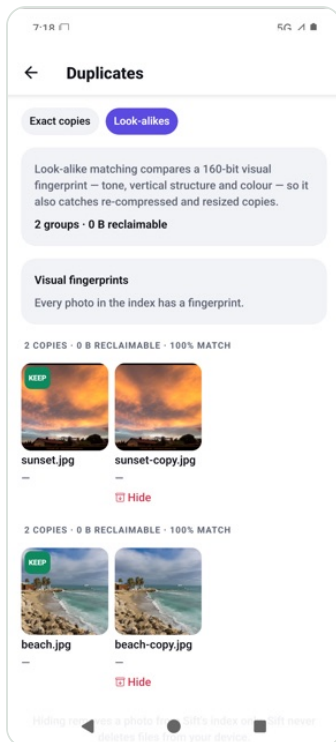
E10 · Task T7: after force-stopping and relaunching the app, the header reads "1 tagged" — the tag survived the restart. ✅



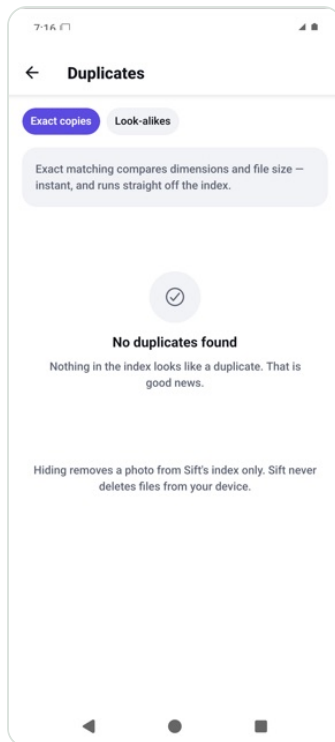
E11 · Task T7: a rescan after two copies were added to the device — the index grew to 18 items without touching existing tags. ✅



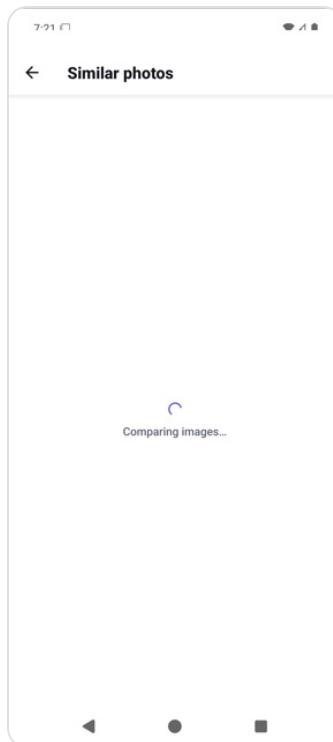
E12 · Task T6: the duplicates screen needs visual fingerprints first and says so, with a one-tap "Fingerprint next 18" prompt.



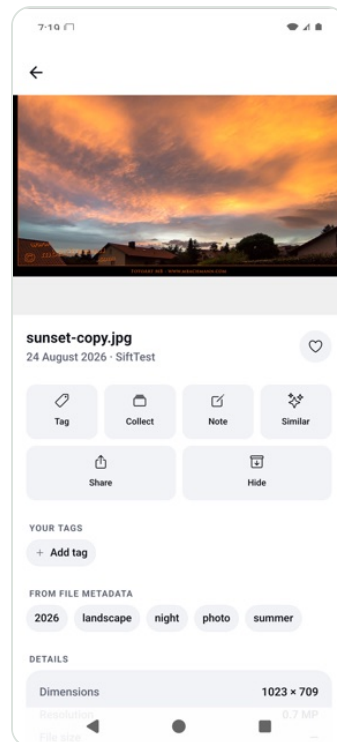
E13 · Task T6 — result: after fingerprinting, both planted duplicate pairs were found at 100% match, and the original in each pair was ranked KEEP. ✅



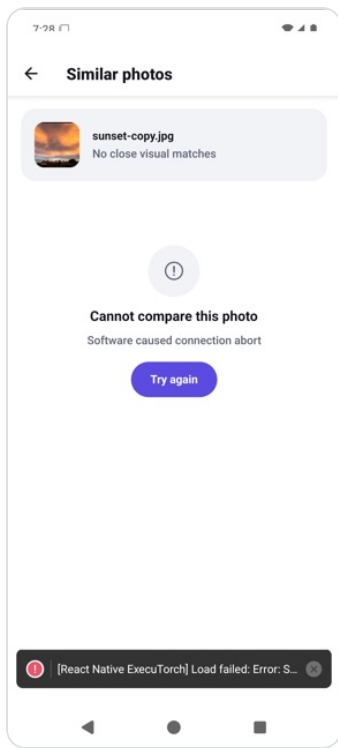
E14 · Task T6 — finding: before fingerprinting, the Exact-copies tab reported "No duplicates found" even though byte-identical copies existed in the index. ⚠️



E15 · Task T5: "Find similar" on a photo — the app shows an explicit "Comparing images..." loading state while it works.



E15a · Task T5: the query photo — the planted sunset copy, whose identical twin exists in the index and should be the top match.



E16 · Task T5 — result: the CLIP model cannot run on an emulator; the app gave a clear error, a Try again button and a load-error toast instead of crashing. ⚠️

Sift · Mobile Applications (UFCF7H-15-3) user-testing component · sessions run 31 August 2026 against commit 66b798d · structure follows the module's sample report (TaskEase).